AI PRINCIPLES & TECHNIQUES

Variable Elimination

Maria Thiele *s1130739*
Angelina Podolyako *s1125886*
Radboud University *December 20, 2024*

Contents

1 Introduction

Calculating the conditional probability of even A given even B is a task that is often required in statistics. Another case, in which such calculations become necessary is when dealing with Bayesian networks. A Bayesian network is a graphical representation of joint probability distributions and can be used to show uncertainty in models (Fusion, 2023). In order to efficiently calculate a conditional probability in a Bayesian network, different approaches can be used. One of these approaches is an algorithm called Variable Elimination. In this assignment, variable elimination is applied on a Bayesian network to calculate the probability of an event given some evidence.

2 Methods

2.1 Specification

The goal of the program is to calculate conditional probabilities through Variable Elimination. The input of the program is a .bif file like the Earthquake network (bnlearn, 2022). The output of the program is a log file, summarizing the key steps that were applied on the data. The basic frame of the program was provided on Kwisthout (2024) in the folder "Assignment 3 Variable Elimination".

2.2 Design

To compute conditional probabilities based on the given Bayesian network in the .bif file, the program performs Variable Elimination. The key steps in Variable Elimination are:

- Conditioning all factors on the evidence
- Eliminate and marginalize each non-query variable
- Normalizing the final factor

In more detail, after the query and the evidence is set, the Variable elimination algorithm first applies the evidence to the other factors. Every row in a factor, that contradicts the evidence, is removed. If the evidence suggests that some property A = true, then every row in which A != true is removed.
After reducing the factors based on the evidence, the algorithm loops through all the variables that need to be eliminated. The order in which the variables are eliminated depends on the heuristic that is being used. For each of these variables, all factors that include this variable are determined. The determined factors are multiplied to generate a new factor. The new factor can then be marginalized, which results in the removal of the variable that needs to be eliminated. With the variable eliminated, the new factor is added to the remaining factors.
After all the variables that need to be eliminated are removed, the final factors are again multiplied to produce the final factor. As a last step, the final factor is normalized, representing the wanted probability.

2.3 Implementation

First, the Bayesian network from the .bif file is converted into dataframes, using the pandas library. This part is done in the read_bayesnet.py file and was already provided. Next, the query and evidence is set and the Variable Elimination algorithm is started. This is done in the file run.py. Additionally, this file prints an overview of the inital probabilities and nodes of the network. Again, the code in this file was provided. The only change that was done in this file is the heuristic for the order of elimination. The nodes are being added to the elimination list by the order of nodes in the network. If a node is either in evidence or in the query, this node is not added. The code for both files can be found in Appendix C or on Kwisthout (2024).

The actual algorithm is implemented in the file variable_elim.py, which can be found in Appendix A. The main operations on the factors are reduction, multiplying and marginalization. The function factor_reduction (lines 137-145) applies the evidence to the other factors by removing rows that become irrelevant due to the evidence being

wrong. Multiplying two factors is done by the function factor_product (lines 147-167). The function takes two dataframes and merges the dataframes into one dataframe. Lastly, marginalization is done with the function factor_marginalization (lines 171-182). The variable that needs to be eliminated is removed from the factor.

The main algorithm uses the three concepts above. After the initializing of the factors (lines 56-59), the evidence is applied on the factors (lines 63-70). The variable elimination part (lines 74-110) first uses factor multiplication and then marginalization to eliminate variables. The final factor is calculated in lines 114-122 and normalized in line 126.

The steps taken are documented in a log file that is updated throughout the runtime of the algorithm. It summarizes the main steps, by displaying the necessary starting information, showing the factors at different times and finally showing the final probability. The log file in Appendix B shows the output for the Earthquake network with the query being "Alarm" and the evidence being "Burglary = True".

3 Testing

The network that was used is the Earthquake network which can be found on bnlearn (2022). The network has five nodes and can be hierarchically represented. A picture of this network is shown in Figure 1 and was copied from the network repository (bnlearn, 2022). The query was set to "Alarm" and the evidence was "Burglary = True". The correct working of the algorithm was checked with a jupyter notebook called VE_pgmpy.ipynb, which can be found in the git repository Kwisthout (2024) and is used for verifying the final probability, and the log file (see Appendix B).
In the log file, intermediate results are shown, like the factors after the evidence is applied (line 46-71).
The testing was done only on this specific network with the given query and evidence. The reasons for that were the available results in the juypter notebook file and the correct form of the .bif file that contained the information about the network.

4 Results

The probability P(Alarm — Burglary = true) is:

- P(alarm — burglary) = 0.9402
- P(¬ alarm — burglary) = 0.0598

The number of factors throughout the runtime is summarized in Table 1.

5 Discussion

During Variable Elimination, the number of factors in this example is constantly decreasing. Due to the elimination of variables, the factors become simpler and are not dependent on many variables.

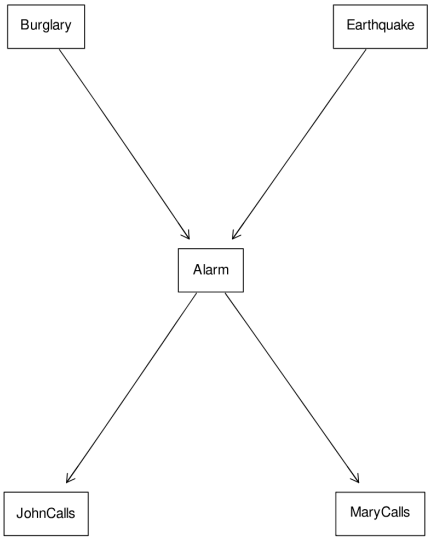

Figure 1: Visual Representation of the Earthquake network

The heuristic used for determining the order for elimination is rather simple. Since the network does not contain many nodes, the approach seems to be straight forward nonetheless. By paying closer attention to the changes in the factors, we can see that the factors MaryCalls&Alarm and JohnCalls&Alarm do not change in their values. This is due to the structure of the network. JohnCalls and MaryCalls are children nodes of the query node and do not influence the probability of P(Alarm—burglary). It would have been more efficient to remove these values first. Nonetheless, due to the small number of nodes, this detail does not mainly improve the efficiency in this example. In other examples, with more nodes and complex structures, removing nodes that do not change the probability that is being searched, leads to higher efficiency and should be considered in the heuristic.

6 Conclusion and Reflection

6.1 Main findings

Through variable elimination conditional probabilities in Bayesian networks can be easily calculated. The use of an appropriate heuristics for the order of variable elimination did not play a central role in the example shown above, but it can have a big influence on the efficiency of the algorithm in larger and more complex networks.

6.2 Limitations and further research

The biggest limitation of this program is the limited generalizability. The input file needs to fit a certain format, otherwise, the network cannot be correctly imported. Furthermore, testing was only performed on a specific example. Other networks, queries or evidence should be considered to verify the correct working of the algorithm in other networks. Lastly, different heuristics for the order of variable elimination should be tested and compared for efficiency.

	Number of factors	List of Factors
Initial factors	5	Burglary Earthquake Alarm and Burglary and Earthquake JohnCalls and alarm MaryCalls and alarm
Updated factors	5	Burglary Earthquake Alarm and Burglary and Earthquake JohnCalls and alarm MaryCalls and alarm
Eliminate Earthquake	4	Burglary Alarm and Burglary JohnCalls and alarm MaryCalls and alarm
Eliminate JohnCalls	3	Burglary Alarm and Burglary MaryCalls and alarm
Eliminate MaryCalls	2	Burglary Alarm and Burglary
Final Factor	1	Burglary and Alarm

Table 1: Summary of Factors over time

6.3 Work division and remarks

The framework of the code was provided by Kwisthout (2024).
Both authors contributed to both coding and writing the report, but Angelina focused mainly on the coding while Maria wrote most of the report.

References

bnlearn. (2022). *Bayesian network repository.* Retrieved December 20, 2024, from https://www.bnlearn.com/bnrepository/

Fusion, B. (2023). *What are bayesian networks?* Retrieved December 20, 2024, from https://www.bayesfusion.com/bayesian-networks/?gad_source=1&gclid=CjwKCAiAyJS7BhBiEiwAyS9uNWHIwk90LUhZWMJd3Udq8RichlN6EdIyCMxOlhmts09iP81PRsdraBwE

Kwisthout, J. H. P. (2024). *Aipt assignments.* GitLab. https://gitlab.socsci.ru.nl/j.kwisthout/aipt-assignments

A Variable Elimination algorithm

```
"""
@Author: Joris van Vugt, Moira Berens, Leonieke van den Bulk

Class for the implementation of the variable elimination algorithm.

"""

import pandas as pd
from datetime import datetime

class VariableElimination():

    def __init__(self, network):
        """
        Initialize the variable elimination algorithm with the specified network.
        Add more initializations if necessary.

        """
        self.network = network

    def run(self, query, observed, elim_order):
        """
        Use the variable elimination algorithm to find out the probability
        distribution of the query variable given the observed variables

        Input:
            query:      The query variable #Q
            observed:   A dictionary of the observed variables {variable: value} #evidence #e
            elim_order: Either a list specifying the elimination ordering
                        or a function that will determine an elimination ordering
                        given the network during the run

        Output: A variable holding the probability distribution
                for the query variable

        """

        # getting the current time to append to the log files name (to identify when the program was run)
        current_time = datetime.now()
        format_1 = current_time.strftime("%H-%M")
        format_2 = current_time.strftime("%H:%M:%S")

        file_name = "VariableEliminationLog_" + format_1

        # Writing the key data points in the log file
        f = open(file_name, "w")
        f.write("Variable Elimination - Log\n\n\n")
        f.write("Run at " + format_2 + "\n\n")
        f.write("Nodes: " + str(self.network.nodes) + "\n")
        f.write("Query: " + query + "\n")
        f.write("Evidence: " + str(observed) + "\n")
        f.write("Elimination order: " + str(elim_order) + "\n")
        f.write("\n\n\n")

        # Initialize the factors and add them to the log
        f.write("Initial Factors: \n\n")
        initialized_factors = list(self.network.probabilities.values())
        for i in initialized_factors:
            f.write(str(i) + "\n\n")
```

```
60
61
62          ########## Reduce the factors based on the evidence and add it to
        the log ##########
63          new_factors = {}
64          for factor_key, factor_val in self.network.probabilities.items():
65              new_factors[factor_key] = self.factor_reduction(factor_val,
        observed)
66
67          f.write("\nUpdated Factors: \n\n")
68          updated_factors = list(new_factors.values())
69          for i in updated_factors:
70              f.write(str(i) + "\n\n")
71
72
73          ########## Multiply and marginalize the factors ##########
74          f.write("Multiplying and marginalizing factors: \n\n")
75          factors_after_multiplying = new_factors.copy()
76
77          for var in elim_order:
78
79              f.write("The variable considered is: " + str(var) + "\n")
80              factors_to_multiply = []
81
82              for factor_key in new_factors.keys():
83                  if var in new_factors[factor_key].columns:
84                      factors_to_multiply.append(new_factors[factor_key])
85                      del factors_after_multiplying[factor_key]
86
87              # Check if there are factors to be multiplied
88              if len(factors_to_multiply) == 0:
89                  f.write("No factors to multiply\n\n")
90                  continue
91              elif len(factors_to_multiply) == 1:
92                  f.write("Only one factor to multiply -> factor does not
        change\n\n")
93              else:
94                  product_factor = factors_to_multiply[0]
95                  for factor in factors_to_multiply[1:]:
96                      product_factor = self.factor_product(product_factor,
        factor)
97
98                  # Apply marginalization
99                  product_factor = self.factor_marginalization(
        product_factor, var)
100
101                 factors_after_multiplying["Muliplied and Marginalized
        Factor"] = product_factor
102
103                 f.write("New factors: \n\n")
104                 for i in factors_after_multiplying.values():
105                     f.write(str(i) + "\n\n")
106
107
108         f.write("\nOverview of the factors after multiplying and
        marginalization is done: \n\n")
109         for i in factors_after_multiplying.values():
110             f.write(str(i) + "\n\n")
111
112
113         ########## Calculate the final factor by multiplying the remaining
         ones ##########
114         f.write("Final Factor: \n\n")
115
116         final_factors = []
117         for factor_val in factors_after_multiplying.values():
```

```
118             final_factors.append(factor_val)
119
120         final_factor = final_factors[0]
121         for factor in final_factors[1:]:
122             final_factor = self.factor_product(final_factor, factor)
123
124
125         ########## Normalize the factor ##########
126         final_factor['prob'] /= final_factor['prob'].sum()
127
128         f.write(str(final_factor))
129         print("\n\nFinal factor:")
130         print(str(final_factor))
131
132         f.close()
133
134
135
136
137     def factor_reduction(self, factor, evidence):
138         """
139         Reduce a factor by applying evidence
140         Every row in a dataframe that is not in accordance with the
    evidence is cut
141         """
142         for var, val in evidence.items():
143             if var in factor.columns:
144                 factor = factor[factor[var] == val]
145         return factor
146
147     def factor_product(self, factor1, factor2):
148         """
149         Multiplies two dataframes and merges them into one dataframe
150         """
151
152         columns_in_both_factors = list(set(factor1.columns) & set(factor2.
    columns))
153
154         if 'prob' in columns_in_both_factors:
155             columns_in_both_factors.remove('prob')
156
157         # Handle two dataframes with no common columns
158         if not columns_in_both_factors:
159             factor1['key'] = 1
160             factor2['key'] = 1
161             new_frame = pd.merge(factor1, factor2, on='key').drop('key',
    axis=1)
162         else:
163             new_frame = pd.merge(factor1, factor2, on=
    columns_in_both_factors, how='outer')
164
165         new_frame['prob'] = new_frame['prob_x'] * new_frame['prob_y']
166         new_frame.drop(['prob_x', 'prob_y'], axis=1, inplace=True)
167         return new_frame
168
169
170     def factor_marginalization(self, factor, variable):
171         """
172         Marginalizes two given dataframes into one
173         """
174
175         remaining_columns = []
176         for col in factor.columns:
177             if col != variable and col!= 'prob':
178                 remaining_columns.append(col)
```

```
180
181         marginalized_factor = factor.groupby(remaining_columns, as_index =
        False)['prob'].sum()
182         return marginalized_factor
```

B Log file

```
1  Variable Elimination - Log
2
3
4  Run at 21:04:54
5
6  Nodes: ['Burglary', 'Earthquake', 'Alarm', 'JohnCalls', 'MaryCalls']
7  Query: Alarm
8  Evidence: {'Burglary': 'True'}
9  Elimination order: ['Earthquake', 'JohnCalls', 'MaryCalls']
10
11
12
13 Initial Factors:
14
15    Burglary  prob
16 0      True  0.01
17 1     False  0.99
18
19    Earthquake  prob
20 0        True  0.02
21 1       False  0.98
22
23     Alarm Burglary Earthquake   prob
24 0    True      True       True  0.950
25 1   False      True       True  0.050
26 2    True     False       True  0.290
27 3   False     False       True  0.710
28 4    True      True      False  0.940
29 5   False      True      False  0.060
30 6    True     False      False  0.001
31 7   False     False      False  0.999
32
33    JohnCalls  Alarm  prob
34 0       True   True  0.90
35 1      False   True  0.10
36 2       True  False  0.05
37 3      False  False  0.95
38
39    MaryCalls  Alarm  prob
40 0       True   True  0.70
41 1      False   True  0.30
42 2       True  False  0.01
43 3      False  False  0.99
44
45
46 Updated Factors:
47
48    Burglary  prob
49 0      True  0.01
50
51    Earthquake  prob
52 0        True  0.02
53 1       False  0.98
54
55     Alarm Burglary Earthquake  prob
56 0    True      True       True  0.95
```

```
1   False      True        True   0.05
4    True      True       False   0.94
5   False      True       False   0.06

   JohnCalls  Alarm  prob
0       True   True  0.90
1      False   True  0.10
2       True  False  0.05
3      False  False  0.95

   MaryCalls  Alarm  prob
0       True   True  0.70
1      False   True  0.30
2       True  False  0.01
3      False  False  0.99

Multiplying and marginalizing factors:

The variable considered is: Earthquake
New factors:

   Burglary  prob
0      True  0.01

   JohnCalls  Alarm  prob
0       True   True  0.90
1      False   True  0.10
2       True  False  0.05
3      False  False  0.95

   MaryCalls  Alarm  prob
0       True   True  0.70
1      False   True  0.30
2       True  False  0.01
3      False  False  0.99

    Alarm Burglary    prob
0   False      True  0.0598
1    True      True  0.9402

The variable considered is: JohnCalls
Only one factor to multiply -> factor does not change

The variable considered is: MaryCalls
Only one factor to multiply -> factor does not change


Overview of the factors after multiplying and marginalization is done:

   Burglary  prob
0      True  0.01

    Alarm Burglary    prob
0   False      True  0.0598
1    True      True  0.9402

Final Factor:

   Burglary  Alarm    prob
0      True  False  0.0598
1      True   True  0.9402
```

C Provided Code

C.1 read_bayesnet.py

```
"""
@Author: Joris van Vugt, Moira Berens, Leonieke van den Bulk

Representation of a Bayesian network read in from a .bif file.

"""

import pandas as pd

class BayesNet():
    """
    This class represents a Bayesian network.
    It can read files in a .bif format (if the formatting is
    along the lines of http://www.bnlearn.com/bnrepository/)

    Uses pandas DataFrames for representing conditional probability tables
    """

    # Possible values per variable
    values = {}

    # Probability distributions per variable
    probabilities = {}

    # Parents per variable
    parents = {}

    def __init__(self, filename):
        """
        Construct a bayesian network from a .bif file

        """
        with open(filename, 'r') as file:
            line_number = 0
            for line in file:
                if line.startswith('network'):
                    self.name = ' '.join(line.split()[1:-1])
                elif line.startswith('variable'):
                    self.parse_variable(line_number, filename)
                elif line.startswith('probability'):
                    self.parse_probability(line_number, filename)
                line_number = line_number + 1

    def parse_probability(self, line_number, filename):
        """
        Parse the probability distribution
        """

        # get line
        line = open(filename, 'r').readlines()[line_number]

        # Find out what variable(s) we are talking about
        variable, parents = self.parse_parents(line)
        next_line = open(filename, 'r').readlines()[line_number + 1].strip
    ()

        # If a variable has no parents, its probabilities start with table
        if next_line.startswith('table'):
            comma_sep_probs = next_line.split('table')[1].split(';')[0].
    strip()
            probs = [float(p) for p in comma_sep_probs.split(',')]
```

```
60              df = pd.DataFrame(columns=[variable, 'prob'])
61              for value, p in zip(self.values[variable], probs):
62                  df.loc[len(df)] = [value, p]
63                  self.probabilities[variable] = df
64          else:
65              #create dataFrame to store the variables
66              df = pd.DataFrame(columns=[variable] + parents + ['prob'])
67
68              #loop over the lines until a line is the same as "}"
69              with open(filename, 'r') as file:
70                  for i in range(line_number + 1):
71                      file.readline()
72                  for line in file:
73                      if '}' in line:
74                          # Done reading this probability distribution
75                          break
76
77                      # Get the values for the parents
78                      comma_sep_values = line.split('(')[1].split(')')[0]
79                      values = [v.strip() for v in comma_sep_values.split(',
    ')]
80
81                      # Get the probabilities for the variable
82                      comma_sep_probs = line.split(')')[1].split(';')[0].
    strip()
83                      probs = [float(p) for p in comma_sep_probs.split(',')]
84
85                  # Create a row in the df for each value combination
86                      for value, p in zip(self.values[variable], probs):
87                          df.loc[len(df)] = [value] + values + [p]
88
89              self.probabilities[variable] = df
90
91      def parse_variable(self, line_number, filename):
92          """
93          Parse the name of a variable and its possible values
94          """
95          variable = open(filename, 'r').readlines()[line_number].split()[1]
96          line = open(filename, 'r').readlines()[line_number+1]
97          start = line.find('{') + 1
98          end = line.find('}')
99          values = [value.strip() for value in line[start:end].split(',')]
100         self.values[variable] = values
101
102     def parse_parents(self, line):
103         """
104         Find out what variables are the parents
105         Returns the variable and its parents
106         """
107         start = line.find('(') + 1
108         end = line.find(')')
109         variables = line[start:end].strip().split('|')
110         variable = variables[0].strip()
111         if len(variables) > 1:
112             parents = variables[1]
113             self.parents[variable] = [v.strip() for v in parents.split(','
    )]
114         else:
115             self.parents[variable] = []
116         return variable, self.parents[variable]
117
118     @property
119     def nodes(self):
120         """Returns the names of the variables in the network"""
121         return list(self.values.keys())
```

C.2 run.py

```
"""
@Author: Joris van Vugt, Moira Berens, Leonieke van den Bulk

Entry point for the creation of the variable elimination algorithm in Python 3.
Code to read in Bayesian Networks has been provided. We assume you have installed the pandas package.

"""
from read_bayesnet import BayesNet
from variable_elim import VariableElimination

if __name__ == '__main__':
    # The class BayesNet represents a Bayesian network from a .bif file in several variables
    net = BayesNet('earthquake.bif') # Format and other networks can be found on http://www.bnlearn.com/bnrepository/

    # These are the variables read from the network that should be used for variable elimination
    print("Nodes:")
    print(net.nodes)
    print("Values:")
    print(net.values)
    print("Parents:")
    print(net.parents)
    print("Probabilities:")
    print(net.probabilities)

    # Make your variable elimination code in the seperate file: 'variable_elim'.
    # You use this file as follows:
    ve = VariableElimination(net)

    # Set the node to be queried as follows:
    query = 'Alarm'

    # The evidence is represented in the following way (can also be empty when there is no evidence):
    evidence = {'Burglary': 'True'}

    # Determine your elimination ordering before you call the run function. The elimination ordering
    # is either specified by a list or a heuristic function that determines the elimination ordering
    # given the network. Experimentation with different heuristics will earn bonus points. The elimination
    # ordering can for example be set as follows:

    elim_order = []
    for i in net.nodes:
        if i == query:
            continue
        elif i in evidence.keys():
            continue
        else:
            elim_order.append(i)

    # Call the variable elimination function for the queried node given the evidence and the elimination ordering as follows:
    ve.run(query, evidence, elim_order)
```